1.5.01 Modeling Practices

Object-Oriented Good Practices: Using private class variables to protect the encapsulation principle of the object-oriented programming.

Conceptual Modeling
- What is it?: Capturing ideas and concepts to allow reasoning, analysis, design and prioritization
- What use it?: To better understand real-world systems, visualize them as-is or as-envisioned, document key decisions, and comunicate efficiently

Conceptual Modeling with UML: Utilizes modeling languages such as use case diagrams, class diagrams, interaction diagrams, and state charts.
Organizing Software into Loosely Coupled Layers: Recommends structuring software in layers.
Prefer Aggregation Over Inheritance: Aggregation is more flexible than inheritance because it can be changed at runtime.

Use Identifiers That Improve Readability and Maintainability:
- Classes should be nouns or noun phrases in Pascal Casing
- Variables should represent the purpose or intended use of an object and use lowerCamelCasing
- Methods perform actions and should use verb verbs or verb phrases.

1.5.07 - Introduction To UML

What is UML?. An expressive, semi-formal, process-independent object-oriented modeling language used to visualize, specify, construct and document software systems.
Building Blocks:
- Elements: Words like classes, objects, interfaces, and states.
- Diagrams: Sentences communicating ideas through structural or behavioral views.
- Views: Chapters describing the system from specific perspective.

Modeling Perspectives:
- Analysis: Focuses on real-world problem domains ("as is")
- Specification: Defines automation boundaries and software requirements.
- Implementation: Focuses on construction using software objects.

Types of Diagrams
- Structural: Class, Object, component, and Deployment diagrams
- Behavioral: Use Case, Sequence, Collaboration, Statechart, and Activity diagrams

Best Practices: Focus on communication, keep layouts clean with minimal crossing lines group elements into packages, and balance structural and behavioral models.

1.05.08 Basic Use Cases

Actors
- Defined as entities outside the system that interact with it, including users other software systems or hardware
- Represented as sets of instances using stick figures, typically named in the singular form to reflect their shared characteristics. Note that while all users are actors, not all actors are users.

Use Cases
- Represent what users need to accomplish, capturing user-visible functionality independently of specific software features or design choices.
- Each use case describes a discrete user goal and acts as a classifier whose instances are specific execution scenarios.

Use Case Diagrams
- Visually document user goals and system functionality using four primary components: Actors, Use Cases, Relationships, and Subjects.

Relationships in Use Cases
- Actor Generalization: Shows that one actor set is a subset of another
- <<Extends>>: Models variations, optional behaviors, or augmentations of a base use case under specific conditions.
- <<Includes>>: Defines how one use case requires the functionality or goal of another.
- Generalization (Use Cases): Defines a specialized use case as a subset of a more general one.

1.05.09 Class Diagrams

- Objects are the fundamental building blocks of object-oriented systems
- Represent any "thing", have a boundary in space and/or time, act as abstractions, encapsulate state and/or behavior, and possess a unique identity.

Introduction to Objects:
- Objects are the fundamental building blocks of object-oriented systems.
- They represent any "thing", have a boundary in space and/or time, act as

Classes
- Classes are abstractions that allow us to manage collections of objects sharing common properties and observable behaviors.
- Class syntax includes compartments for name, attributes, operations, and responsibilities

Attributes
- Represent characteristics or properties holding values or references to other objects.
- Syntax: follows
  [visibility] name [multiplicity] [type] [=initial-value]
  [{property-string}]

Escaneado con CamScanner

Operations
- Define ways objects interact (analysis), service obligations (specification), or member methods/functions (implementation)
- Syntax uses lowerCamelCase verb phrases with parameter lists and return types.

Student
major: String gpa: Real standing: String
add(Class Section) drop(Class Section)

1.05.10 - UML - Quick Reference Guide

General Analysis Best Practices:
- Maintain an "analysis perspective" focus on the real-world problem or stakeholder requirements without worrying about software components yet.
- Let incomplete diagrams drive further questions and exploration, treating the entire model like an essay where each diagram is a paragraph.
- Keep diagrams readable by avoiding crossing lines and using color effectively.

Use Case Diagrams
- Actors: External entities (users, systems, hardware) with goals regarding the system.
- Use Cases: User-visible goals or functionalities.
- Relationships: Includes (<<include>>), extends (<<extend>>), and generalization/specialization.

Class Diagrams
- Classes and Objects: Real-world or application domain concepts represented as sets of objects.
- Relationships: generalization/specialization, named binary associations, N-ary associations, aggregation, composition, and multiplicity constraints
- Attributes: Object properties or data placeholders.

What are They Used for?
1. Visualizing System Structure: They provide a clear blueprint of the system's architecture, showing what objects exist, what data they hold, and how they relate to one another.
2. Supporting Different Development Perspectives.
   - Analysts: To understand the real-world problem domain and map domain concepts
   - Specification: To define software interfaces, object boundaries and public behaviors
   - Implementation: To serve as a direct guide for writing code and structuring classes in object-oriented programming languages.

Activity Diagrams
- Model procedural flow, control flows, object flows, decision nodes, fork/join concurrent threads, and signal actions using initial and final nodes

Interaction Diagrams
- Model dynamic objects perspectives, communication channels, synchronous/asynchronous messages, object creation/destruction, and execution occurrences.

State Charts
- Model object lifecycle behavior using states, initial/final states, and guarded/triggered transitions with actions ([Guard] Trigger / Action), along with internal/entry/exit activities.

Escaneado con CamScanner